SheetAgent

An autonomous AI agent that takes one natural-language instruction and generates employee data, drives Microsoft Excel, writes the same data to Google Sheets, verifies both, and reports what happened.

Submitted by

Repository	github.com/Harryj05/sheetagent1
Model provider	Gemini (gemini-3.5-flash) by default; Anthropic supported
Tests	102 passing - no network, no API key, no Excel required
Platform	Windows for the real-Excel path; Linux/CI runs headless

How the agent works

A normal script is told **what steps to do**. This agent is told **what tools exist** and decides the steps itself. That is why the same program handles "just make a CSV" (one tool) and "do everything" (four tools) without a line of code changing.

1. You type one sentence. The agent has four tools: make a CSV, import into Excel, import into Google Sheets, and verify both.
2. The request and the tool catalogue go to the model. The model returns an ordered **plan**, which is printed before anything runs.
3. Any step naming a tool that does not exist is discarded, so a hallucinated step can never execute.
4. The executor loop begins: the model asks for a tool, the runtime runs it, and the result goes back to the model, which decides what to do next.
5. If a tool fails, the model is told it failed and why. The run continues with the steps that still work rather than crashing.
6. The tools do real work - Excel actually opens and saves the workbook; Google Sheets is written through the official Sheets API v4.
7. A **separate** tool then verifies: it re-opens the saved workbook and re-reads the live sheet, comparing both against the source CSV. The agent may not simply claim success.
8. Permanent failures (missing credentials) are not retried; transient ones (rate limits, Excel busy) get exponential backoff.
9. A final report lists every step as SUCCESS, FAILED or PARTIAL, with file paths and the spreadsheet URL.

Interactive prompts

Run any of these with `python -m sheetagent "<prompt>"`. Nothing else is required after the command. Prompts marked **[verified live]** have been executed end to end against the real Excel application and the live Google Sheets API.

1. The assignment prompt - full workflow [verified live]

```
Create a sample employee CSV and import it into Excel and Google Sheets.
```

All four tools run: generate, Excel, Sheets, verify. In the recorded run the model planned four steps, chose 25 rows on its own initiative, drove Excel through COM, wrote 25 rows to the live sheet, and verification confirmed the row count and all nine headers (`all_verified: true`).

2. Explicit row count

```
Generate 50 realistic employee records, import them into Excel, upload them to Google
Sheets, and confirm both imports succeeded.
```

Demonstrates that quantities are taken from the request, not from a constant.

3. CSV only - proves tool selection is dynamic

```
Just generate 25 employee records as a CSV file. Don't open Excel or touch Google Sheets.
```

One tool call instead of four. Same code, same registry - the model read the request and chose. This is the clearest evidence the workflow is not hardcoded.

4. Excel only

```
Make some employee data and get it into Excel. Skip Google Sheets entirely.
```

Two tools. The Google Sheets tool is never invoked.

5. Alternate spreadsheet format

```
Create 30 employee records, save the workbook as an .ods file, and also upload the data to
Google Sheets.
```

Exercises the XLSX / XLSM / CSV / ODS support in the Excel tool.

6. Write into an existing spreadsheet

```
Regenerate the employee data with 40 rows and write it into the existing spreadsheet
1AbCdEfGhIjKlMnOpQrStUvWxYz.
```

Overrides the configured spreadsheet id for a single run.

7. Memory across runs

```
Run 1: Create an employee CSV and import it into Excel. Run 2: Now upload the CSV you made
last time to Google Sheets.
```

The second run resolves the path from persisted memory rather than asking.

8. Error handling, on purpose [verified live]

```
Import /tmp/definitely-not-here.csv into Excel and Google Sheets.
```

The tool returns `status: failed` with the reason and a hint; the agent reports the failure rather than inventing success. Verified live: the report read *import_csv_to_excel: FAILED - CSV not found*.

9. Verification only

```
Check whether output/employees.xlsx and the Google Sheet still match output/employees.csv.
```

A single tool call that re-reads both targets and compares them to the CSV.

Requirements coverage

Functional requirements

Requirement	Status
Accept natural language input	Met
Decide which tools to execute	Met - model-driven, verified live
Generate a CSV automatically	Met
At least 20 rows of realistic data	Met - 20+ rows, 9 columns
Launch Microsoft Excel	Met - real COM, Excel 16.0
Import the CSV into Excel	Met
Save the workbook	Met - frozen header, numeric salaries
Connect via the Google Sheets API	Met - Sheets API v4, verified live
Import the same CSV into a Google Sheet	Met
Report whether each step succeeded	Met
Handle errors gracefully	Met

Bonus items

Item	Notes
Multi-step planning	Separate planner stage; unknown tools dropped
Memory / conversation history	Facts kept; transcript bounded to a constant
XLSX / CSV / ODS	Plus XLSM
Retry logic	Classified per error class in both integrations
Configurable tools	enabled_tools hides a tool from the model entirely
MCP server	Four tools exposed; schemas derived from the registry
Dockerized deployment	Builds; scoped honestly as a CI / headless runner
Unit tests	102, fully offline
Structured logging	JSONL, one object per line, run-id tagged
Progress updates	Live event feed during execution

Beyond the brief: a second model provider (Gemini) behind an adapter, and GitHub Actions CI running the suite on Ubuntu with no Excel and no credentials present.

Quick start

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
set GEMINI_API_KEY=your-key
python -m sheetagent "Create a sample employee CSV and import it into Excel and Google
Sheets."
```

Full setup, including both Google authentication flows, is in the repository README.

Two limitations worth stating plainly

A Google service account cannot create spreadsheets. It has no Drive storage quota, so it must be pointed at a sheet you created and shared with it. With OAuth desktop credentials the agent creates sheets itself. The README documents both.

The Docker image cannot launch Excel. COM requires Windows and an installed Excel, so the image is scoped as a CI and headless test runner. Every headless run reports `excel_launched: false` rather than implying Excel ran.